البرمجة الغرضية التوجه IPG 203

للفصل الدراسي S25

SVU

الجامعة الافتراضية السورية

SYRIAN VIRTUAL UNIVERSITY

الطلاب المشاركون:

1) ahmed_320964

2) alisar_328755

3) baidaa_317802

4) sedra_324211

5) thanaa_321540

تحت إشراف الدكتور:

T – Hwaida ALDaoud

IPG

OOP

تُعد البرمجة الغرضية التوجه (OOP) الركيزة الأساسية في تطوير البرمجيات الحديثة، حيث تتيح بناء أنظمة معقدة تتميز بالمرونة وقابلية التوسع وإعادة الاستخدام. يأتي هذا المشروع، المطور بلغة **#C**ضمن ملف **"Program.cs"**، كنموذج تطبيقي حي لهذه المبادئ من خلال بناء **"نظام إدارة الموظفين (Employee Management System)"**.

يهدف التطبيق إلى محاكاة بيئة عمل حقيقية يتم فيها إدارة فئات متنوعة من الموظفين داخل الأقسام المؤسسية، مع التركيز على أتمتة العمليات الحسابية والرقابية مثل حساب الرواتب والتحقق من صحة البيانات وإصدار التنبيهات الذكية .تم تصميم الكود ليكون مثالاً نموذجياً لكيفية تحويل المتطلبات الإدارية إلى هيكلية برمجية متماسكة تعتمد على التجريد والتغليف وتعدد الأشكال .

شرح التطبيق :

يقوم التطبيق المنفذ في **"Program.cs"**بإدارة الهيكل الوظيفي لشركة تقنية، ويتلخص عمله في النقاط التالية:

1.الهيكل التنظيمي(Structural Hierarchy)

- **blueprintالموظف :**يعتمد النظام على فئة مجردة (Employee) تعمل كقاعدة أساسية لجميع أنواع الموظفين، مما يضمن وجود خصائص مشتركة مثل الاسم، العمر، والراتب الأساسي .
- **تخصيص الأدوار :**يتيح النظام إنشاء أنواع محددة من الموظفين عبر الوراثة، وهم (المدير Managerوالمطور Developerوالمتدرب .Intern)
- **إدارة الأقسام :**توفر فئة Departmentوعاءً تنظيمياً يجمع الموظفين ويسمح بمعالجتهم ككتلة واحدة باستخدام قوائم ديناميكية .(List<Employee>)

2.العمليات الوظيفية(Core Operations)

- **حساب الرواتب الديناميكي :**يمتلك التطبيق قدرة على حساب الراتب بشكل مختلف لكل فئة؛ فالمدير يضاف له حافز(Bonus) ، والمطور يُحسب راتبه بناءً على ساعات العمل ومعدل الأجر بالساعة، بينما يتقاضى المتدرب راتباً ثابتاً .
- **نظام التحقق من البيانات :**يتضمن التطبيق فئة ساكنة DataValidatorتعمل كرقيب على البيانات المدخلة، حيث تمنع إدخال أسماء غير صالحة أو أعمار خارج النطاق القانوني (١٨-٦٥ عاماً) .

- **تتبع الإحصائيات العامة :**يقوم النظام آلياً بتحديث إحصائيات إجمالية تشمل العدد الكلي للموظفين وإجمالي ميزانية الرواتب في الشركة عبر خصائص ساكنة .(Static Properties)

3.الرقابة والتنبيهات(Monitoring & Alerts)

- **نظام التنبيه الذكي :**يستخدم التطبيق "التفويضات والأحداث & Delegates) " (Eventsلمراقبة الرواتب العالية .بمجرد أن يتجاوز راتب أي موظف سقف **10,000$**، يقوم النظام تلقائياً بإطلاق حدث تنبيهي يظهر للمسؤول في شاشة التحكم .

4.عرض النتائج(Reporting)

- عند تشغيل البرنامج في **"Program.cs"**، يقوم بإنشاء قسم تقني (Technology Department)، ويضيف إليه موظفين من مختلف الفئات، ثم يستعرض مهام كل موظف وبياناته المالية بوضوح، مما يظهر قوة "تعدد الأشكال (Polymorphism) "في التعامل مع كائنات مختلفة عبر واجهة برمجية موحدة .

مبادئ OOP الأربعة في الكود:

1.التجريد(Abstraction)

يتمثل في فصل "ماذا يفعل الكائن" عن "كيفية القيام به."

- **IEmployeeActionsواجهة :(Interface) –** تحدد العمليات الأساسية التي يجب أن يقوم بها أي موظف (CalculateSalary, DisplayInfo, Work) دون الدخول في تفاصيل التنفيذ.
- **Employeeفئة مجردة :(Abstract Class) –** لا يمكنك إنشاء كائن (Object) مباشرة من Employeeلأنه مجرد نموذج عام.
- **الدوال المجردة :**الدالتان CalculateSalary()و Work()تم تعريفهما كـ abstract داخل فئة الموظف، مما يجبر الفئات الوارثة على تقديم التنفيذ الخاص بها.

```
// تعريف الواجهة (Interface)
public interface IEmployeeActions
{
    double CalculateSalary();
    void Work();
}

// تعريف الفئة المجردة (Abstract Class)
public abstract class Employee : IEmployeeActions
{
    public abstract double CalculateSalary(); // دالة
مجردة
    public abstract void Work();
}
```

.التغليف(Encapsulation)

يتمثل في حماية البيانات ومنع الوصول المباشر إليها إلا من خلال وسيط (Properties).

- **الحقول الخاصة (Private Fields):** مثل _name, _age, _baseSalary. لا يمكن الوصول إليها من خارج الفئة.
- **الخصائص (Properties):** مثل Name, Age, BaseSalary. تلاحظ وجود منطق برمجي (Logic) داخل الـ set للتأكد من صحة البيانات باستخدام الفئة DataValidator.
- **الوصول المقيد:** الخاصية Id لها private set مما يجعلها "للقراءة فقط" من الخارج بعد تعيينها في المنشئ.(Constructor)

```
private int _age;
public int Age
{
    get { return _age; }
    private set
    {
        if (!DataValidator.IsValidAge(value))
            throw new ArgumentException("Age must be
between 18 and 65!");
        _age = value;
    }
                                                        }
```

الوراثة(Inheritance).

تسمح لفئة بأن ترث الخصائص والدوال من فئة أخرى لتجنب تكرار الكود.

- **الفئات التابعة:** Manager, Developer, و Internجميعها ترث من الفئة الأساسية Employee.
- **إعادة الاستخدام:** جميع هذه الفئات حصلت تلقائياً على خصائص الاسم والعمر والـ ID ودالة DisplayInfo()دون الحاجة لإعادة كتابتها.

```
public class Manager : Employee
{
    public Manager(int id, string name, int age, double
baseSalary, double bonus)
        : base(id, name, age, baseSalary) //  استدعاء
منشئ الفئة الأم
    {
        Bonus = bonus;
    }
                                                        }
```

.تعدد الأشكال(Polymorphism)

يسمح للكائنات المختلفة بأن تُعامل كنوع واحد، ولكن كل كائن يستجيب بطريقته الخاصة.

- **الكلمة المفتاحية: override**تم استخدامها في Manager, Developer, وIntern لإعادة تعريف دالتي CalculateSalary() وWork().
- **التطبيق العملي:** في فئة Department وبالتحديد في دالة ShowAllEmployees()، يتم التعامل مع قائمة من نوع List<Employee>. عندما نستدعي emp.Work()، يقوم البرنامج بتنفيذ نسخة الدالة الخاصة بنوع الموظف الفعلي المدير يطبع"managing" ، والمطور يطبع("writing code").

```
foreach (var emp in _employees)
{
    emp.Work(); // كل موظف سيعمل بطريقته الخاصة (مدير، مطور، متدرب)
    emp.DisplayInfo();
}
```

شرح دور التفويض (Delegates) والأحداث(Events) :

تستخدم هذه الأدوات لتطبيق نمط "المراقب(Observer Pattern) "، وهو باختصار وسيلة لإخبار أجزاء أخرى من البرنامج بحدوث شيء معين.

.1التفويضSalaryAlertHandler: (Delegate)

دوره: هو "عقد" أو "توقيع" يحدد شكل الدالة التي يمكنها التعامل مع التنبيه. هو يقول: "أي دالة تريد استقبال تنبيه الراتب العالي يجب أن تأخذ نصاً (اسم الموظف) ورقماً (الراتب)."

.2الحدثOnHighSalary: (Event)

- **دوره:** هو الوسيلة التي يرسل بها الكائن إشارة عند تحقق شرط معين. في الكود، يتم إطلاق هذا الحدث داخل دالة CheckSalary() إذا تجاوز الراتب ١٠,٠٠٠.
- **الاشتراك: (Subscription)** في دالة Main تلاحظ السطر m1.OnHighSalary += HandleHighSalaryAlert. هنا قمنا بربط الحدث بالدالة التي ستعالجه.

```
// تعريف التفويض والحدث
public delegate void SalaryAlertHandler(string
employeeName, double salary);
public event SalaryAlertHandler OnHighSalary;

// إطلاق الحدث عند تحقق الشرط
if (salary > 10000)
{
    OnHighSalary?.Invoke(Name, salary);
}
```

الأعضاء الساكنة (Static Members) :

Employee.Count: : خاصية ساكنة تُستخدم لحساب عدد الموظفين الكلي في النظام، وهي مشتركة بين كل الكائنات وليست خاصة بموظف واحد.

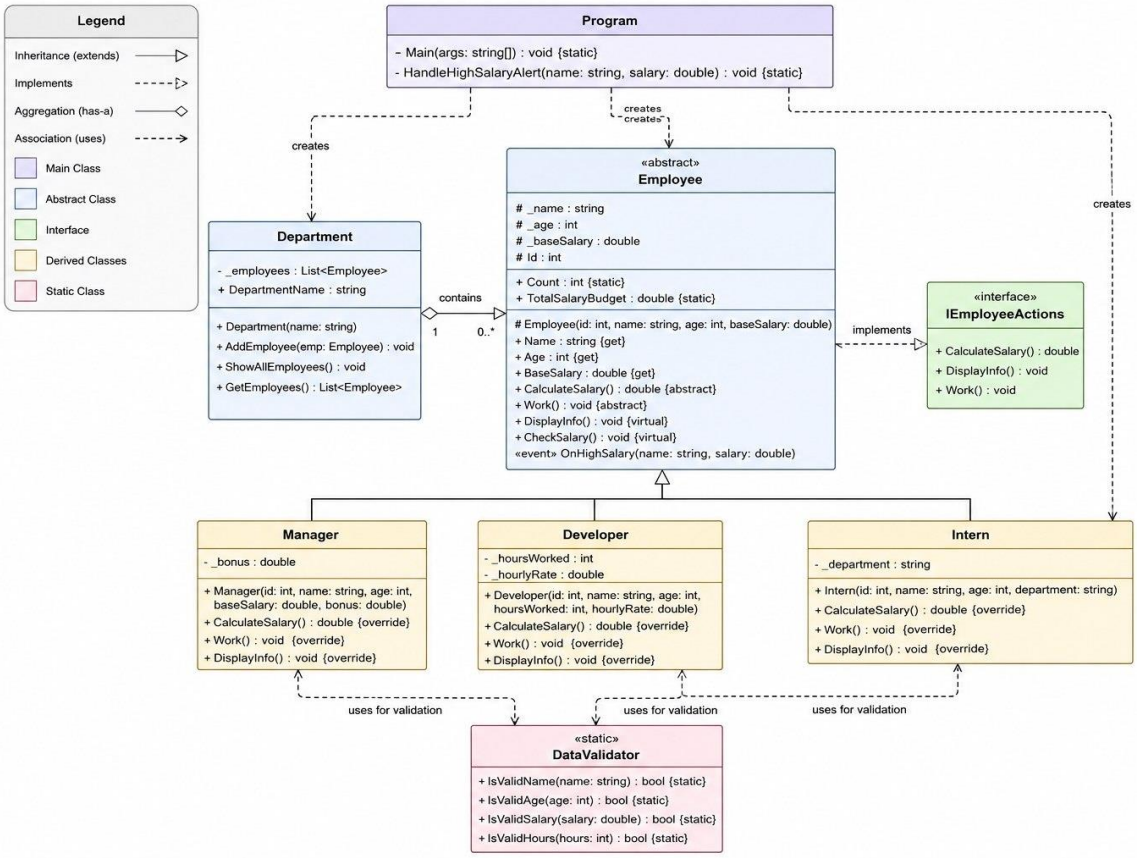
DataValidator: : فئة ساكنة بالكامل(Static Class) ، لا نحتاج لإنشاء كائن منها. تُستخدم كأداة مساعدة (Utility) للتحقق من صحة البيانات في أي مكان بالكود.

```
public static class DataValidator
{
    public static bool IsValidAge(int age)
    {
        return age >= 18 && age <= 65;
    }
}
```

ملخص الهيكل البرمجي

المفهوم	الأماكن في الكود
Interfaces	`IEmployeeActions`
Abstract Class	`Employee`
Derived Classes	`Manager` , `Developer` , `Intern`
Logic/Validation	`DataValidator`
Encapsulation	`private` fields + `public` properties with logic
Polymorphism	`() override CalculateSalary()` , `override Work`
Events	`OnHighSalary` , `HandleHighSalaryAlert`

UML



https://github.com/ahmedaljbhjy5-svg/IPG203-HW-E.M.S